LLM Conversation (2 conversations) (using Drafter planning tool LLM):

I ran the code, and it runs in Thonny. You do not have to run the play function. Please make sure the program activates drafter and launches a website, and add unit tests when applicable
Worked for 7s
Since your program already runs in Thonny, the important change is to make main.py start the Drafter website rather than run the old console-based play() function.
The version below:
Starts Drafter with start_server(...).
Includes the website routes and pages.
Includes unit tests for the calculations and helper functions.
Runs the tests before launching the website.
Does not call play().
main.py
Replace your current main.py with this:

```
from dataclasses import dataclass
from drafter import *
import final_project
@dataclass
class State:
"""Store the information used by the Budget Tracker website."""
budget: float
transactions: list[final_project.Transaction]
names: list[str]
money_amounts: list[float]
percents: list[float]
entry_finished: bool
def get_total_spent(state: State) -> float:
"""Calculate the total amount spent on all transactions."""
total_spent = 0.0
for transaction in state.transactions:
total_spent += transaction.money
return total_spent
def get_remaining_budget(state: State) -> float:
"""Calculate the amount of money remaining in the budget."""
return state.budget - get_total_spent(state)
def update_percentages(state: State) -> None:
"""Calculate and save the percentage for each transaction."""
state.transactions = final_project.calculate_percents(
state.transactions,
state.budget
)
state.percents = []
```

```
for transaction in state.transactions:
state.percents.append(transaction.percent)
def make_summary(state: State) -> list:
"""Create the content for the transaction summary page."""
content = [
Header("Budget Summary"),
"Starting budget: $" + format(state.budget, ".2f"),
Header("Your Transactions")
]
count = 0
for transaction in state.transactions:
count += 1
content.append(
str(count) + ". " + transaction.name +
" - $" + format(transaction.money, ".2f") +
" (" + format(transaction.percent, ".2f") + "%)"
)
total_spent = get_total_spent(state)
remaining = get_remaining_budget(state)
content.extend([
Header("Financial Summary"),
"Total spent: $" + format(total_spent, ".2f"),
"Remaining budget: $" + format(remaining, ".2f")
])
return content
```

Before running it

Keep main.py and final_project.py in the same folder.

Make sure final_project.py defines Transaction, check_transactions, and calculate_percents.

Make sure final_project.py does not call play() automatically at the bottom. It should not request console input when main.py imports it.

Replace "Replace with your name" in set_site_information.

Run main.py in Thonny, not final_project.py.

When it runs, the Shell should print All unit tests passed! before Drafter starts. Drafter should then start its website server; use the local website address it reports to open the app in your browser.

One important note: These unit tests check the calculation and helper functions. They do not automatically test every browser interaction or route, so you should still click through the website and test invalid inputs, an empty transaction list, and spending over budget.

This version keeps the minimalist design from your planning document and does not add graphs, editing, or deleting features.

This might also interest you.

something is still making the code run in thonny and not a website

I think the key distinction is that Drafter is launched by running your Python file, but the actual app should appear in a web browser. Running main.py from Thonny is normal; Thonny is just where the Drafter server starts.
However, if you're only seeing Python output in the Shell and no website, we need to check how Drafter is being launched. The start_server(...) call should be the final step in your program—not a call to play().
I also spotted a likely cause in your original final_project.py: it ends with:
play()
Even though main.py doesn't call play(), importing final_project runs that line automatically. Delete that line (or comment it out). Otherwise, Thonny will start the old console program when Drafter imports the module.
Make these changes
In final_project.py, remove the standalone play() at the bottom.
In main.py, keep the Drafter launch as the final line, but make it explicit:
start_server(
State(0.0, [], [], [], [], False),
open_browser=True,
site_title="Budget Tracker"
)
In Thonny, open and run main.py, not final_project.py.
Leave the Thonny Shell running. Drafter uses it to host your local development site. Your browser should open automatically. If it doesn't, visit **http://localhost:8000** manually.
Drafter Documentation
+1
One thing I need to know
When you run main.py, what happens?
Does Thonny ask you to enter a budget or transaction?
Does it print All unit tests passed! and then stop?
Does it show a URL or an error message?
If you paste the exact Shell output here, I can pinpoint the problem rather than guessing.
This might also interest you.
i removed the play() call, but the code still runs in the thonny shell
That's actually expected up to a point: Thonny's Shell is where the Drafter server runs. Drafter doesn't replace Thonny or move your Python program out of the Shell. It uses the Shell to start a local web server, and you interact with the app in a browser.
The important question is whether the server starts successfully and gives you a browser address.
According to Drafter's current documentation, running your Python file is the normal way to start the website. It starts a local development server, usually at http://localhost:8000, and opens the browser automatically.
Drafter Documentation
+1
Try this first
At the very bottom of main.py, replace your existing start_server(...) call with:

start_server(
State(0.0, [], [], [], [], False),
open_browser=True,
site_title="Budget Tracker",
port=8000
)
Then:
Run main.py in Thonny.
Leave the Shell open.
Open Chrome, Firefox, or another browser.
Go to **http://localhost:8000**.
If that doesn't work
Please tell me what happens when you run the file:
A. The Shell says the server is running, but no browser opens.
B. The Shell prints All unit tests passed! and then stops.
C. The Shell displays an error.
D. The browser opens, but the page is blank or shows an error.
That will help me identify whether the issue is the Drafter server, browser launch, or your application code.
This is what appeared in the shell: >>> %Run final_project_drafter.py
TEST PASSED - [line 55] assert_equal(check_transactions([], 1.0), "No Transactions!")
TEST PASSED - [line 56] assert_equal(check_transactions([Transaction("poop", 2.0, 0.0)], 1.0), "Budget exceeded!")
TEST PASSED - [line 57] assert_equal(check_transactions([Transaction("pee", 1.0, 0.0)], 1.0), "")
TEST PASSED - [line 71] assert_equal(calculate_percents([], 0.0), [])
TEST PASSED - [line 72] assert_equal(calculate_percents([Transaction("poop", 10.0, 0.0), Transaction("pee", 20.0, 0.0)], 100.0), [Transaction("poop", 10.0, 10.0), Transaction("pee", 20.0, 20.0)])
Enter a budget (a float):
I found the problem! Your Shell output tells us exactly what's happening:
>>> %Run final_project_drafter.py
...
Enter a budget (a float):
Thonny is running final_project_drafter.py, which is still running your original console-based budget program. It is not starting the Drafter website.
Here's how to fix it
1. Stop the current program.
Click the red Stop button in Thonny.
2. Open final_project_drafter.py.
Look at the top of the file. The Drafter version should begin with:
from dataclasses import dataclass
from drafter import *
import final_project

It should contain your State dataclass and the @route functions.
3. Check the bottom of the file.
It should end with something like:
start_server(
State(0.0, [], [], [], [], False)
)
It should not end with play().
4. Save and run final_project_drafter.py again.
Keep the Shell open and visit http://localhost:8000 in your browser if it doesn't open automatically.
One important question
Do you have two separate files in Thonny?
final_project.py — your original transaction calculations.
final_project_drafter.py — your new website code.
If you do, that's the correct arrangement. You need to make sure the second file contains the Drafter code, not the original console program.
Context window full
Start a new chat with an automatic summary of this conversation, or upgrade to continue here with more context available in every chat.
2nd LLM Conversation:
Read this conversation:
I ran the code, and it runs in Thonny. You do not have to run the play function. Please make sure the program activates drafter and launches a website, and add unit tests when applicable
Worked for 7s
Since your program already runs in Thonny, the important change is to make main.py start the Drafter website rather than run the old console-based play() function.
The version below:
Starts Drafter with start_server(...).
Includes the website routes and pages.
Includes unit tests for the calculations and helper functions.
Runs the tests before launching the website.
Does not call play().
main.py
Replace your current main.py with this:
```
from dataclasses import dataclass
from drafter import *
import final_project
@dataclass
class State:
"""Store the information used by the Budget Tracker website."""
budget: float
transactions: list[final_project.Transaction]
names: list[str]
money_amounts: list[float]
```

```
percents: list[float]
entry_finished: bool
def get_total_spent(state: State) -> float:
"""Calculate the total amount spent on all transactions."""
total_spent = 0.0
for transaction in state.transactions:
total_spent += transaction.money
return total_spent
def get_remaining_budget(state: State) -> float:
"""Calculate the amount of money remaining in the budget."""
return state.budget - get_total_spent(state)
def update_percentages(state: State) -> None:
"""Calculate and save the percentage for each transaction."""
state.transactions = final_project.calculate_percents(
state.transactions,
state.budget
)
state.percents = []
for transaction in state.transactions:
state.percents.append(transaction.percent)
def make_summary(state: State) -> list:
"""Create the content for the transaction summary page."""
content = [
Header("Budget Summary"),
"Starting budget: $" + format(state.budget, ".2f"),
Header("Your Transactions")
]
count = 0
for transaction in state.transactions:
count += 1
content.append(
str(count) + ". " + transaction.name +
" - $" + format(transaction.money, ".2f") +
" (" + format(transaction.percent, ".2f") + "%)"
)
total_spent = get_total_spent(state)
remaining = get_remaining_budget(state)
content.extend([
Header("Financial Summary"),
"Total spent: $" + format(total_spent, ".2f"),
"Remaining budget: $" + format(remaining, ".2f")
])
return content
Before running it
```

Keep main.py and final_project.py in the same folder.
Make sure final_project.py defines Transaction, check_transactions, and calculate_percents.
Make sure final_project.py does not call play() automatically at the bottom. It should not request console input when main.py imports it.
Replace "Replace with your name" in set_site_information.
Run main.py in Thonny, not final_project.py.
When it runs, the Shell should print All unit tests passed! before Drafter starts. Drafter should then start its website server; use the local website address it reports to open the app in your browser.
One important note: These unit tests check the calculation and helper functions. They do not automatically test every browser interaction or route, so you should still click through the website and test invalid inputs, an empty transaction list, and spending over budget.
This version keeps the minimalist design from your planning document and does not add graphs, editing, or deleting features.
This might also interest you.
something is still making the code run in thonny and not a website
I think the key distinction is that Drafter is launched by running your Python file, but the actual app should appear in a web browser. Running main.py from Thonny is normal; Thonny is just where the Drafter server starts.
However, if you're only seeing Python output in the Shell and no website, we need to check how Drafter is being launched. The start_server(...) call should be the final step in your program—not a call to play().
I also spotted a likely cause in your original final_project.py: it ends with:
play()
Even though main.py doesn't call play(), importing final_project runs that line automatically. Delete that line (or comment it out). Otherwise, Thonny will start the old console program when Drafter imports the module.
Make these changes
In final_project.py, remove the standalone play() at the bottom.
In main.py, keep the Drafter launch as the final line, but make it explicit:
start_server(
State(0.0, [], [], [], [], False),
open_browser=True,
site_title="Budget Tracker"
)
In Thonny, open and run main.py, not final_project.py.
Leave the Thonny Shell running. Drafter uses it to host your local development site. Your browser should open automatically. If it doesn't, visit **http://localhost:8000** manually.
Drafter Documentation
+1
One thing I need to know
When you run main.py, what happens?
Does Thonny ask you to enter a budget or transaction?
Does it print All unit tests passed! and then stop?

Does it show a URL or an error message?
If you paste the exact Shell output here, I can pinpoint the problem rather than guessing.
This might also interest you.
i removed the play() call, but the code still runs in the thonny shell
That's actually expected up to a point: Thonny's Shell is where the Drafter server runs. Drafter doesn't replace Thonny or move your Python program out of the Shell. It uses the Shell to start a local web server, and you interact with the app in a browser.
The important question is whether the server starts successfully and gives you a browser address.
According to Drafter's current documentation, running your Python file is the normal way to start the website. It starts a local development server, usually at http://localhost:8000, and opens the browser automatically.
Drafter Documentation
+1
Try this first
At the very bottom of main.py, replace your existing start_server(...) call with:
start_server(
State(0.0, [], [], [], [], False),
open_browser=True,
site_title="Budget Tracker",
port=8000
)
Then:
Run main.py in Thonny.
Leave the Shell open.
Open Chrome, Firefox, or another browser.
Go to **http://localhost:8000**.
If that doesn't work
Please tell me what happens when you run the file:
A. The Shell says the server is running, but no browser opens.
B. The Shell prints All unit tests passed! and then stops.
C. The Shell displays an error.
D. The browser opens, but the page is blank or shows an error.
That will help me identify whether the issue is the Drafter server, browser launch, or your application code.
This is what appeared in the shell: >>> %Run final_project_drafter.py
TEST PASSED - [line 55] assert_equal(check_transactions([], 1.0), "No Transactions!")
TEST PASSED - [line 56] assert_equal(check_transactions([Transaction("poop", 2.0, 0.0)], 1.0), "Budget exceeded!")
TEST PASSED - [line 57] assert_equal(check_transactions([Transaction("pee", 1.0, 0.0)], 1.0), "")
TEST PASSED - [line 71] assert_equal(calculate_percents([], 0.0), [])

TEST PASSED - [line 72] assert_equal(calculate_percents([Transaction("poop", 10.0, 0.0), Transaction("pee", 20.0, 0.0)], 100.0), [Transaction("poop", 10.0, 10.0), Transaction("pee", 20.0, 20.0)])
Enter a budget (a float):
I found the problem! Your Shell output tells us exactly what's happening:
>>> %Run final_project_drafter.py
...
Enter a budget (a float):
Thonny is running final_project_drafter.py, which is still running your original console-based budget program. It is not starting the Drafter website.
Here's how to fix it
1. Stop the current program.
Click the red Stop button in Thonny.
2. Open final_project_drafter.py.
Look at the top of the file. The Drafter version should begin with:
from dataclasses import dataclass
from drafter import *
import final_project
It should contain your State dataclass and the @route functions.
3. Check the bottom of the file.
It should end with something like:
start_server(
State(0.0, [], [], [], [], False)
)
It should not end with play().
4. Save and run final_project_drafter.py again.
Keep the Shell open and visit http://localhost:8000 in your browser if it doesn't open automatically.
One important question
Do you have two separate files in Thonny?
final_project.py — your original transaction calculations.
final_project_drafter.py — your new website code.
If you do, that's the correct arrangement. You need to make sure the second file contains the Drafter code, not the original console program.
Context window full
Start a new chat with an automatic summary of this conversation, or upgrade to continue here with more context available in every chat.
…
I've read the entire conversation. I can see the problem, and your latest Shell output gives us an important clue.
The issue is not Thonny itself. Your original console-based program is still being executed somewhere.
Your Shell shows:
>>> %Run final_project_drafter.py

TEST PASSED ...
Enter a budget (a float):
This tells us three things:
Thonny is running final_project_drafter.py.
Your original unit tests are executing successfully.
Something is still calling input() to request a budget, meaning the original console functionality is still being executed.
Removing play() was not necessarily enough. Your original final_project.py might contain other instructions that execute automatically when imported.
Let's fix this without guessing.
Please send me these two things:
Your complete final_project.py file (the original budget calculations and unit tests).
Your complete final_project_drafter.py file (the website code).
You can paste both directly into the chat using separate Python code blocks.
I'll check them to identify:
What is causing the console program to launch.
Whether Drafter is being initialized correctly.
Whether your website routes are configured correctly.
Whether your unit tests can run without interfering with the website.
My goal is to get your Budget Tracker opening in a web browser at http://localhost:8000, while keeping your existing calculations and applicable unit tests intact.
I won't unnecessarily rewrite your project or remove features you've already implemented.
This might also interest you.
Pasted text(2).txtDocument
Here is my code from final_project_drafter.
I've reviewed your entire final_project_drafter.py. Good news: your Drafter file appears to be set up correctly to launch a website. It already contains the important components:
@route functions for navigating the website.
A State dataclass to store budget and transaction information.
Unit tests that run before the website launches.
A start_server() call with open_browser=True and port 8000.
However, I found something important.
The likely source of the problem
On line 3, you have:
import final_project
When Python imports another file, it executes that file's top-level instructions.
Your previous Shell output showed:
TEST PASSED - [line 55] ...
TEST PASSED - [line 57] ...
Enter a budget (a float):
Those TEST PASSED messages are not produced by the unit tests in your Drafter file. They are coming from your original final_project.py.
This strongly suggests that your original file is still executing its console-based instructions when Drafter imports it.

What I need next
Please upload your complete final_project.py file.
I want to inspect:
Where the console-based budget input is being requested.
Whether your original unit tests are running automatically upon import.
Whether your play() function or another function is being called automatically.
I suspect we can fix this with a small change to final_project.py rather than rewriting your Drafter website.
Don't change anything in final_project_drafter.py yet. Your website code already has the correct general structure, and I want to preserve the work you've done.
Here is final_project.py from dataclasses import dataclass
from bakery import assert_equal
@dataclass
class Transaction:
name: str
money: float
percent: float
def get_budget() -> float:
'''takes a budget float value as an input from the user'''
budget = float(input("Enter a budget (a float): "))
return budget
def get_amount() -> float:
'''prompts the user for a float and returns that float'''
return float(input("Enter a money amount (float); "))
def get_name() -> str:
'''prompts the user for a string and returns that string'''
return str(input("Enter a transaction name (STOP to stop): "))
def make_transactions() -> list[Transaction]:
'''Takes a name string value and a money float value from the user as inputs
until the user enters STOP, then turns the values into a list of transactions,
with 0.0 as a placeholder value for the percent'''
name = ""
value = 0.0
names = []
values = []
while name != "STOP":
name = get_name()
if name != "STOP":
value = get_amount()
names.append(name)
values.append(value)
transactions = []
count = 0
for name in names:

```
transactions.append(Transaction(name, values[count], 0.0))
return transactions
def check_transactions(transactions: list[Transaction], budget: float) -> str:
'''takes a list of transactions and a budget (float), returning "No Transactions!" if
the transaction list is empty, or returning "Budget exceeded!" if the total money
spent exceeds the budget. Returns an empty string otherwise.'''
if not transactions:
return "No Transactions!"
total_spent = 0.0
for transaction in transactions:
total_spent += transaction.money
if total_spent > budget:
return "Budget exceeded!"
return ""
assert_equal(check_transactions([], 1.0), "No Transactions!")
assert_equal(check_transactions([Transaction("poop", 2.0, 0.0)], 1.0), "Budget exceeded!")
assert_equal(check_transactions([Transaction("pee", 1.0, 0.0)], 1.0), "")
def calculate_percents(transactions: list[Transaction], budget: float) -> list[Transaction]:
'''consumes a list of Transactions and a budget (float), returning a new list of
Transactions with the 0.0 placeholder precent value replaced with the percent of
the budget that transaction's money value is'''
new_list = []
percent = 0.0
if budget == 0.0:
return []
for transaction in transactions:
percent = (transaction.money / budget) * 100
new_list.append(Transaction(transaction.name, transaction.money, percent))
return new_list
assert_equal(calculate_percents([], 0.0), [])
assert_equal(calculate_percents([Transaction("poop", 10.0, 0.0), Transaction("pee", 20.0, 0.0)],
100.0), [Transaction("poop", 10.0, 10.0), Transaction("pee", 20.0, 20.0)])
def play():
'''Generates the transaction list and displays it, with the budget'''
budget = get_budget()
transactions = calculate_percents(make_transactions(), budget)
if check_transactions(transactions, budget) == "":
print("Budget: " + str(budget) + "\n")
count = 0
for transaction in transactions:
count += 1
print(str(count) + ": Name: " + str(transaction.name) +", Amount: $", str(transaction.money) + ",
Percent of budget: " + str(transaction.percent) + "% \n")
else:
```

return check_transactions(transactions, budget)
#play()
#above line would run this code in the shell if uncommented
…
I've reviewed your final_project.py, and I found something important: your original file is actually correct in one very important respect.
Your play() function is commented out:
#play()
#above line would run this code in the shell if uncommented
That means importing final_project.py should NOT automatically launch your console-based program.
1. What is happening?
When final_project_drafter.py runs this line:
import final_project
Python executes the top-level code in final_project.py.
That includes your unit tests:
assert_equal(check_transactions([], 1.0), "No Transactions!")
assert_equal(check_transactions([Transaction("poop", 2.0, 0.0)], 1.0), "Budget exceeded!")
This explains why your Shell displays the TEST PASSED messages.
However, nothing in the final_project.py you provided calls get_budget() or requests console input.
Therefore, the prompt:
Enter a budget (a float):
cannot be explained by the code you've just provided.
2. What I want you to check
I suspect Thonny might be executing an older or different version of your file.
Please try these steps:
Stop the current program using Thonny's red Stop button.
Open final_project.py.
Press Ctrl + S to save it.
Open final_project_drafter.py.
Press Ctrl + S again.
Make sure both files are saved in the same folder.
Run final_project_drafter.py directly.
3. One more important check
Use Thonny's file editor to search your project for:
play()
Also search for:
get_budget()
Look for any active calls that aren't inside a function definition.
What I expect to happen
Expected behavior
Your Shell should display the unit test results, followed by a message indicating that Drafter's server has started.

Your browser should then open the Budget Tracker website at:
http://localhost:8000
You should see your welcome page with the Get Started button.
One question before we make further changes: When you run final_project_drafter.py, does the Shell still display Enter a budget (a float): after the unit tests, even after completing the steps above?
I don't want to unnecessarily change working code when the problem may simply be that Thonny is executing a different file.
it works good now thanks for the help
You're very welcome! I'm glad we got your Budget Tracker working in Drafter.
Good luck with your final project! 🎉